المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport de Travaux Pratiques

TP2 -- Monolithe Modulaire

Restructuration et Preparation a la Migration Microservices

Realisee par	Nawar El Asraoui
Filiere	Cybersecurite et Intelligence Artificielle
Encadre par	Pr. BE ELBAGHAZAOUI
Annee universitaire	2025 -- 2026

Mars 2026

1. Introduction et Objectifs

Ce TP2 a pour objectif de **restructurer le monolithe du TP1** en modules metier distincts avec des boundaries claires, d'introduire les design patterns **DTO, Mapper et Service Layer**, et d'implementer des interfaces de communication inter-modules pour preparer une future migration vers une architecture microservices. Le projet du TP1 sert de base et est etendu avec deux nouveaux modules : **Customer** et **Order**.

1.1 Correspondance objectifs / realisation

Objectifs du TP	Realisation dans le projet
Restructurer le monolithe en modules metier distincts	3 modules : product, customer, order avec packages dedies
Definir des boundaries claires entre domaines	Chaque module possede ses propres model, dto, service, controller
Implementer des interfaces de communication inter-modules	CustomerPublicService et ProductPublicService
Appliquer les design patterns (DTO, Mapper, Service Layer)	Records Java comme DTOs, MapStruct pour mapping, interface + impl
Preparer le terrain pour une future migration microservices	Les modules communiquent uniquement via interfaces publiques

2. Nouvelle Structure Modulaire

La restructuration du monolithe consiste a organiser l'ensemble du code selon une **architecture modulaire par domaine fonctionnel**. Chaque module possede ses propres sous-packages : model, dto, mapper, repository, service et controller. Cette organisation garantit qu'un module ne peut acceder aux donnees d'un autre module que via son interface publique, jamais directement via son repository.

2.1 Modules et classes principales

Module	Package	Classes principales
Product	product/	Product, Category, ProductService, ProductController, ProductDTO, CreateProductRequest, UpdateProductRequest, ProductMapper
Customer	customer/	Customer, CustomerService, CustomerPublicService, CustomerPublicServiceImpl, CustomerController, CustomerDTO, CreateCustomerRequest, CustomerMapper

Order	order/	Order, OrderItem, OrderService, OrderController, OrderDTO, OrderItemDTO, CreateOrderRequest, OrderItemRequest, OrderMapper

2.2 Design patterns appliques

Conformement aux objectifs du TP, cinq design patterns ont ete appliques systematiquement :

Design Pattern	Classe / Interface	Role
DTO	ProductDTO, CustomerDTO, OrderDTO...	Decouplage couche HTTP / persistance
Mapper (MapStruct)	ProductMapper, CustomerMapper, OrderMapper	Conversion automatique entite <-> DTO
Service Layer	ProductService, CustomerService, OrderService	Centralisation de la logique metier
Interface publique	CustomerPublicService, ProductPublicService	Communication inter-modules sans couplage fort
Repository	ProductRepository, CustomerRepository, OrderRepository	Acces aux donnees via Spring Data JPA

2.3 Diagramme de dependances entre modules

Le sujet impose un diagramme de dependances clair. L'architecture implementee respecte exactement le schema demande : le **Product Module** et le **Customer Module** sont independants l'un de l'autre et n'ont aucune dependance externe. Le **Order Module** depend des services publics des deux autres modules (via leurs interfaces), mais **jamais de leurs repositories directement**. Cette contrainte architecturale est le coeur du TP.

- Product Module --> gere uniquement les produits, aucune dependance externe
- Customer Module --> gere les clients, completement independant
- Order Module --> depend de CustomerPublicService et ProductPublicService uniquement

3. Interfaces de Communication Inter-Modules

La nouveaute architecturale majeure du TP2 est l'introduction d'**interfaces publiques** pour la communication entre modules. Plutot que d'injecter directement un service d'un autre module (couplage fort), chaque module expose une interface minimaliste qui represente son **contrat public**. Cette approche est directement inspiree du principe de separation des contextes bornes (Bounded Context) des architectures microservices.

Interface	Module source	Methodes exposees
CustomerPublicService	customer	existsCustomer(Long id) : boolean
ProductPublicService	product	existsProduct(Long id), getProductPrice(Long id), getProductName(Long id)

Chaque interface est implementee par une classe dediee (**CustomerPublicServiceImpl** et **ProductPublicServiceImpl**). Le module Order injecte uniquement ces interfaces, garantissant que si demain Customer devient un microservice, seule l'implementation de l'interface change (appel HTTP), pas le code du OrderService.

4. Implementation des Modules Customer et Order

4.1 Module Customer

Le module Customer implemente les trois operations demandees par le sujet. Le **CustomerService** gere la liste des clients, la recherche par ID et la creation. La creation integre une **validation d'unicite de l'email** via repo.existsByEmail() avant toute insertion. La methode **existsCustomer(Long id)** requise par le sujet est implementee dans CustomerService et exposee via l'interface CustomerPublicService, consommee par le module Order.

4.2 Module Order

Le module Order est le plus complexe. Le **OrderService** orchestre la creation d'une commande en validant d'abord l'existence du client, puis pour chaque ligne d'article, l'existence du produit. Le prix unitaire et le nom du produit sont captures au moment de la commande et stockes dans **OrderItem**, garantissant un historique fiable independamment des futures modifications du catalogue.

Flux de creation d'une commande :

- Verification de l'existence du client via CustomerPublicService.existsCustomer()
- Creation de l'entite Order avec customerId et date courante (LocalDateTime.now())
- Pour chaque item : verification produit, recuperation prix via **getProductPrice()** et nom via **getProductName()**
- Construction de chaque OrderItem avec prix et nom captures a l'instant T
- Calcul du total : accumulation de (prix x quantite) pour chaque item
- Persistance et retour du OrderDTO complet avec ses items

L'endpoint d'historique demande par le sujet est implemente : **GET /api/customers/{id}/orders** retourne les commandes d'un client triees par date decroissante via la methode derivee Spring Data **findByCustomerIdOrderByCreatedAtDesc()**.

5. DTOs, Validations et Regles Metier

5.1 DTOs implementes

DTO / Record	Module	Champs et validations
CreateCustomerRequest	customer	fullName (@NotBlank), email (@NotBlank @Email)
CustomerDTO	customer	id, fullName, email
CreateOrderRequest	order	customerId (@NotNull), items (@Size min=1)
OrderItemRequest	order	productId (@NotNull), quantity (@NotNull @Positive)
OrderDTO	order	id, customerId, createdAt, total, items (List<OrderItemDTO>)
OrderItemDTO	order	productId, productName, quantity, unitPrice

5.2 Regles metier implementees

Regle metier	Service	Mecanisme
Email client unique	CustomerService	repo.existsByEmail() avant creation
Client existant pour commande	OrderService	CustomerPublicService.existsCustomer()
Produit existant par ligne	OrderService	ProductPublicService.existsProduct()
Prix capture au moment de la commande	OrderService	getProductPrice() -> stocke dans OrderItem
Nom produit capture a la commande	OrderService	getProductName() -> stocke dans OrderItem
Total calcule automatiquement	OrderService	Accumulation prix x quantite par item

6. Endpoints REST

L'application ecoute sur le port 8081. Le tableau suivant recapitule l'ensemble des 13 endpoints exposes apres le TP2 :

Methode	URL	Description
GET	/api/products	Lister tous les produits
GET	/api/products/{id}	Obtenir un produit par ID
POST	/api/products	Creer un produit

PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit
GET	/api/categories	Lister les categories
POST	/api/categories	Creer une categorie
GET	/api/customers	Lister tous les clients
GET	/api/customers/{id}	Obtenir un client par ID
POST	/api/customers	Creer un client (email unique)
POST	/api/orders	Creer une commande
GET	/api/customers/{id}/orders	Historique des commandes d'un client

6.1 Exemples de requetes

Creer un client -- POST /api/customers :

{ "fullName": "Nawar El Asraoui", "email": "nawar@ensa.ma" }

Creer une commande -- POST /api/orders :

{ "customerId": 1, "items": [{ "productId": 1, "quantity": 2 }, { "productId": 3, "quantity": 1 }] }

Historique des commandes -- GET /api/customers/1/orders :

Retourne la liste triee par date decroissante des commandes du client 1, avec leurs items, prix unitaires et total calcule.

7. Preparation a la Migration Microservices

Le TP2 constitue une **etape de transition architecturale** deliberee. En introduisant les interfaces publiques inter-modules, l'application simule dans un monolithe les contraintes qui existent naturellement en microservices. Chaque module est deja concu comme s'il pouvait etre deploye independamment.

7.1 Avantages de la restructuration

- **Separation des responsabilites :** chaque module possede un perimetre fonctionnel bien defini, ce qui facilite la maintenance et l'evolution.
- **Faible couplage :** le module Order ne connait pas l'implementation interne de Customer ni de Product, seulement leurs contrats publics.
- **Testabilite :** les interfaces publiques peuvent etre facilement mockees dans les tests unitaires.
- **Evolutivite :** si un module migre vers un microservice, seule son implementation d'interface change, pas le code des autres modules.

7.2 Limites residuelles du monolithe

- **Base de donnees partagee :** tous les modules utilisent la meme instance H2. Un echec affecte l'ensemble du systeme.
- **Deploiement uniforme :** impossible de deployer uniquement le module Order sans redeployer toute l'application.
- **Scalabilite uniforme :** un module tres sollicite ne peut pas etre scale independamment des autres.

8. Conclusion

Ce TP2 a permis de restructurer l'application monolithique du TP1 en appliquant une organisation modulaire rigoureuse et les design patterns DTO, Mapper et Service Layer. L'implementation des interfaces publiques CustomerPublicService et ProductPublicService represente la contribution architecturale majeure : elle etablit des boundaries claires entre modules et prepare concretement la migration vers une architecture microservices. Les trois exercices demandes par le sujet ont ete realises : les modules Customer et Order sont implementes avec toutes leurs couches, la methode de verification d'existence du client est en place, et l'endpoint d'historique des commandes par client est fonctionnel. Le TP3 exploitera ces acquis en deploying ces modules comme services distributes independants.